

CSCI316 · AUTUMN 2026

Predicting Loan Default at Origination

An end-to-end big-data mining workflow on 855,969 Lending Club loans

Stage 1 — Apache Spark data exploration

Stage 2 — Spark MLlib + TensorFlow / Keras

Target: default_ind · 5.43% default (17.4 : 1) · no post-origination feature used as a predictor

HEADLINE RESULT

0.183

best PR-AUC (DeepMLP) — 3.35× the 5.46% random baseline

0.802

ROC-AUC, TensorFlow MLP and DeepMLP

55%

of defaults recovered by the single-unit LogisticNN at its tuned threshold

The Problem: Who Will Default?

Task. Binary classification of `default_ind` — 1 if the borrower stopped paying, 0 otherwise.

Constraint that shapes everything. The prediction must be usable at the moment the loan is approved.

Why that matters. The dataset carries 13 post-funding columns (`total_pymnt`, `recoveries`, `last_pymnt_amnt`, `out_prncp` ...). They describe repayment behaviour, so a model trained on them scores near-perfectly and is worthless in production.

Business framing. A false negative is a funded loan that is written off; a false positive is forgone interest. The two errors are not symmetric, so we rank borrowers by risk rather than hand back a hard yes / no.

PROJECT OBJECTIVE

Build a reproducible, leakage-free workflow that ranks loan applications by default risk, and compare two learning stacks on the same features and the same split.

SUCCESS IS MEASURED BY

- PR-AUC — the ranking metric that survives a 17 : 1 imbalance
- Recall and precision on the default class at a tuned threshold
- ROC-AUC, reported for comparability
- Accuracy, reported but never used to choose a model

A constant “no default” rule already scores 94.53% accuracy. Accuracy alone cannot separate a good model from a useless one.

The Dataset at a Glance

855,969

loans, verified against the raw CSV line count

73 → 48

columns after cleaning; 38 enter the models

5.43%

default rate — 46,467 defaults, 17.4 : 1

0

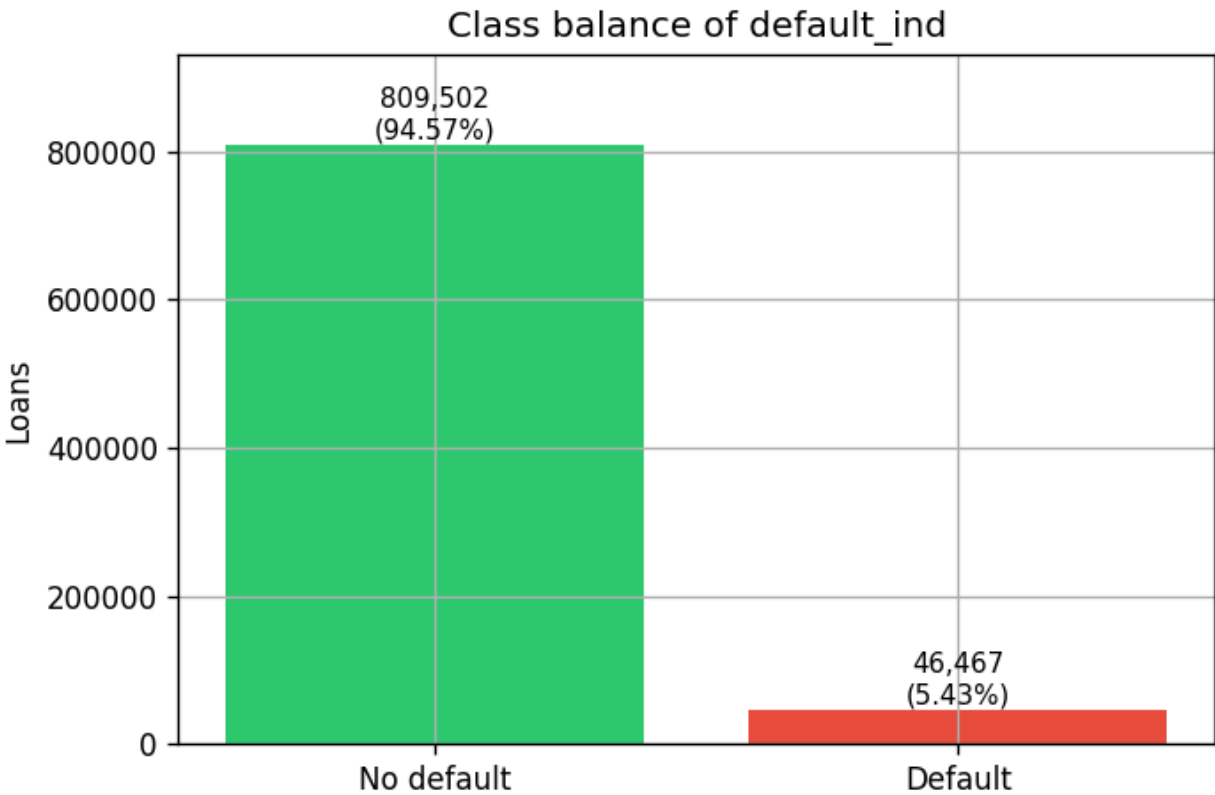
rows lost — 100% retained after cleaning

Mixed types. Numeric, categorical and date columns; several arrive as text — '36 months', '83.7%', dd-MM-yyyy — and are cast explicitly.

Sparsity. 42 of 73 columns are fully populated; 21 are more than half empty (the joint-application and instalment-line blocks) and were dropped.

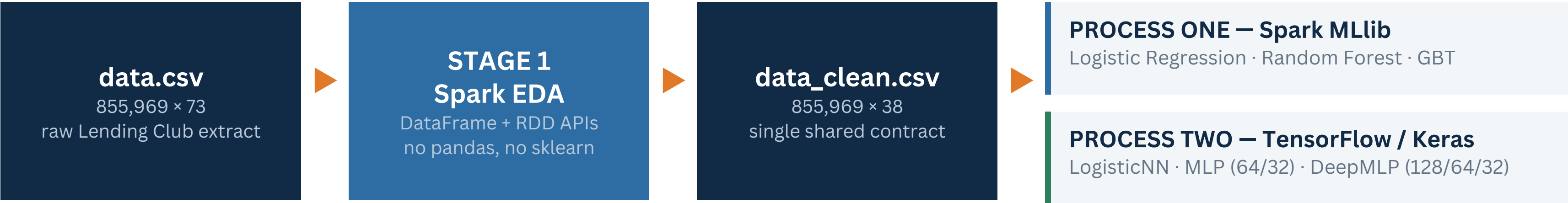
Keys and constants. id and member_id are unique across all 855k rows; policy_code has a single distinct value; pymnt_plan is degenerate.

Quality fixes. revol_util capped at 150 and dti at 100 to tame implausible tails; no duplicate rows and no missing targets were found.



Stage 1 — class balance of default_ind

End-to-End Workflow



WHY THIS SHAPE

One cleaning pass, two stacks Stage 1 writes a single canonical file. Both Stage 2 processes read it, so any difference in the results comes from the model, not the preprocessing.	Identical split protocol Stratified 64 / 16 / 20, class weights from the training fold only, threshold tuned on validation, test touched exactly once.	Spark where the data is big Stage 1 runs over all 855,969 rows in Spark. Keras needs a dense in-memory matrix, so Stage 2 works on a shared 30% stratified sample.	Reproducible by construction Fixed seeds, timestamped run folders, a runs.log and a pinned final/ directory that every figure in this deck comes from.
--	---	---	---

01

Stage 1 — Data Exploration

Apache Spark DataFrame and RDD APIs only · no pandas, no scikit-learn

How We Explored 855,969 Loans

1

Parser sanity check

Compare the parsed record count against a raw RDD line count so that a mis-quoted CSV cannot silently drop rows.

2

Integrity

Duplicate rows, duplicate ids, null counts for all 73 columns, and a per-column schema dump.

3

Typing

'36 months' → 36, '83.7%' → 83.7, dd-MM-yyyy → date, then derived ratios: loan_to_income, installment_to_income, bal_to_limit.

4

Statistics

Summary statistics, class balance, class-conditional means and Pearson correlations with the target.

5

Categorical risk

Default rate by grade, sub_grade, term, purpose, home_ownership, verification_status, emp_length and addr_state.

6

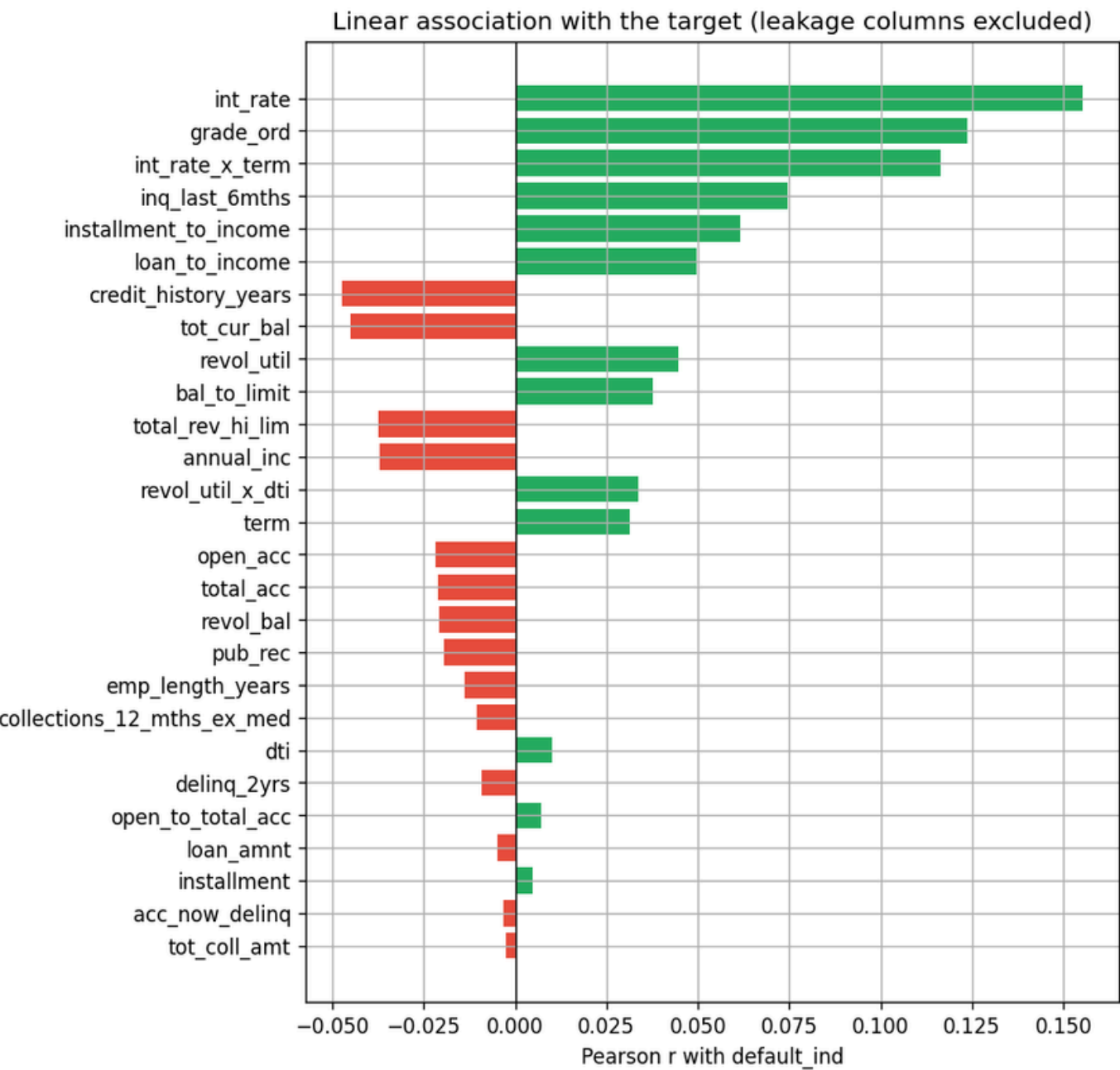
Selection

The 7 most and 7 least relevant attributes, each with a written justification, plus the leakage exclusion list.

`src/stage1_edu.py – parser sanity check`

```
# Spark only – the assignment forbids pandas / sklearn in Stage 1
raw_lines = spark.sparkContext.textFile(str(csv_path)).count() - 1
assert df.count() == raw_lines, 'the CSV parser lost or invented rows'
```

What the Numbers Say



Pearson correlation of each numeric attribute with default_ind

Interest rate dominates. int_rate is the single strongest numeric signal, $r = +0.155$. Defaults were priced at a mean 16.02% versus 13.03% for the rest, a 22.9% gap.

Everything else is weak. No other attribute passes $|r| = 0.13$. The relationships are individually weak and non-linear, which is exactly the regime where trees and networks earn their keep over a linear rule.

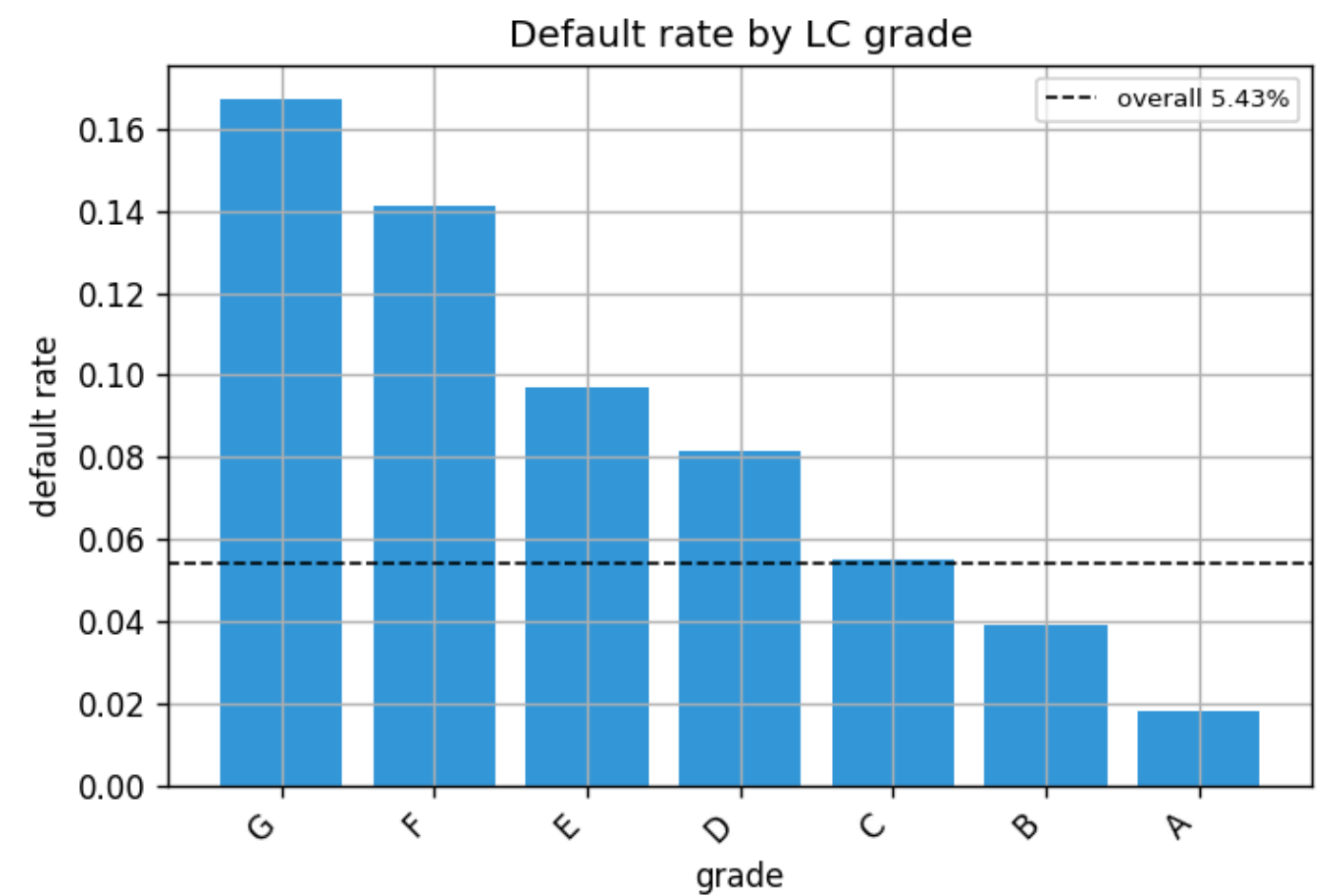
Affordability shows up as ratios. Raw annual_inc reaches only $r = -0.037$, but the derived loan_to_income lifts it to $+0.050$ and installment_to_income to $+0.062$ — feature engineering, not new data.

Thin files default more. credit_history_years $r = -0.047$ and inq_last_6mths $r = +0.074$: short histories and recent credit-seeking both point the same way.

CAVEAT WE RETURN TO

int_rate is Lending Club's own risk price. It is legitimately available at origination, but it means part of what we predict is what the lender already concluded.

Risk Is Concentrated in the Credit Grade



Default rate by assigned credit grade (n = 855,969)

Grade	Loans	Default rate
A	145,665	1.83%
B	247,998	3.92%
C	236,855	5.49%
D	132,802	8.13%
E	66,448	9.72%
F	21,328	14.13%
G	4,873	16.70%

Monotonic, 9× spread. 1.83% at grade A to 16.70% at grade G — the cleanest separation in the data.

Term adds exposure. 60-month loans default at 6.52% against 4.96% for 36-month loans.

Purpose is noisy. 'educational' is the riskiest stated purpose at 17.18% — but on only 326 loans.

ENCODING DECISION

Grade is ordinal, so we encode it as `grade_ord` rather than as seven unordered dummies — the model gets the ordering for free.

The Seven Most Relevant Attributes

#	Attribute	Evidence	Justification
1	int_rate	$r = +0.155$	Strongest single numeric signal — the risk-based price Lending Club charges summarises its own underwriting view.
2	grade / sub_grade	9× spread	Default rate climbs monotonically from 1.83% (A) to 16.70% (G): a 14.88-point spread across seven bands.
3	term	$r = +0.031$	1.55-point spread between the two terms; a 60-month loan is exposed for twice as long as a 36-month one.
4	dti	$r = +0.010$	The standard measure of repayment capacity, and it is available at application time.
5	revol_util	$r = +0.045$	Revolving utilisation shows how much headroom the borrower has already consumed.
6	inq_last_6mths	$r = +0.074$	Recent credit enquiries proxy active credit seeking — an early distress signal.
7	annual_inc	$r = -0.037$	Income anchors every affordability ratio; the derived loan_to_income reaches $r = +0.050$.

Selected on evidence, not intuition: correlation with the target for numeric attributes, and the spread in group default rate for categorical ones — both computed in Spark over all 855,969 rows.

The Seven Least Relevant – and the Ones We Banned

#	Attribute	Why it was excluded
1	id	Surrogate loan key, unique for all 855k rows – pure noise, usable only through row order.
2	member_id	Surrogate borrower key. Identifiers must never enter a credit model.
3	policy_code	Constant – a single distinct value, so it carries exactly zero information.
4	pymnt_plan	Degenerate: fewer than 10 loans take the minority value, so it cannot generalise.
5	emp_title	Free text with tens of thousands of inconsistent spellings; unusable without NLP.
6	title	Borrower-typed loan title that merely restates purpose, which is already encoded.

SEPARATELY BANNED – TARGET LEAKAGE

13 columns that only exist after the loan is funded:

`out_prncp · out_prncp_inv · total_pymnt · total_pymnt_inv · total_rec_prncp · total_rec_int · total_rec_late_fee · recoveries · collection_recovery_fee · last_pymnt_d · last_pymnt_amnt · next_pymnt_d · last_credit_pull_d`

These are not weak features – they are the answer. recoveries is non-zero only for loans that already defaulted. Including them would produce a near-perfect model that cannot score a single new application.

21 sparse columns (the joint-application and instalment-line blocks, all >50% empty) and 4 free-text columns were dropped as well: 73 columns in, 48 out, 38 into the models.

02

Stage 2 — Predictive Analysis

Two independent processes, one shared feature contract and one shared split

The Evaluation Protocol Both Processes Share

Stratified 64 / 16 / 20

Each class is split separately, so train, validation and test all carry the same 5.46% default rate. Verified and written out as `split_summary.csv`.

Train-only fitting

Median imputation, string indexing, one-hot encoding and standardisation are fitted on the training fold and applied to the other two. Unseen categories encode to an all-zero block.

Class weights, not resampling

Every default counts as $n_{\text{neg}} / n_{\text{pos}} \approx 17.3$ rows. Nothing is duplicated or discarded, so the test set stays an honest sample of reality.

Threshold tuned on validation

0.5 is arbitrary under imbalance. We sweep $0.05 \rightarrow 0.95$ and keep the threshold that maximises F1 on the default class — chosen on validation, never on test.

Test used once

Model selection, hyperparameters and the threshold are all settled before the test set is scored a single time.

PR-AUC ranks the models

With 5.46% positives, ROC-AUC flatters everything and accuracy is dominated by the majority class. PR-AUC is the metric we select on.

`stage2_spark.py` – `add_class_weights()`

```
# every default counts as (n_negative / n_positive) rows
ratio = negatives / positives      # 17.31 on the train fold
df = df.withColumn('weight', F.when(
    F.col('default_ind') == 1.0, F.lit(ratio)
).otherwise(F.lit(1.0)))
```

`best_threshold()` – identical in both processes

```
# F1 on the default class, searched on VALIDATION data
for t in np.arange(0.05, 0.951, 0.005):
    f1 = metrics_at(y_val, s_val, t)['f1_default']
    if f1 > best_f1:
        best, best_f1 = t, f1
```

Process One — Spark MLlib

Pipeline. Imputer (median) → StringIndexer → OneHotEncoder → VectorAssembler → classifier, assembled as one pyspark.ml Pipeline and fitted once on the training fold.

Design matrix. 138 assembled columns from 27 numeric and 6 categorical inputs, plus a bureau_data_missing indicator.

Tuning. CrossValidator with 3 folds, selecting on areaUnderPR — not on accuracy.

Engineering issue we hit. handleInvalid='keep' on both the indexer and the encoder: a category present only in validation or test would otherwise abort the run instead of encoding to zeros.

Model	Selected hyperparameters
Logistic Regression	regParam 0.001, elasticNetParam 0.0 (pure L2)
Random Forest	numTrees 100, maxDepth 12
Gradient-Boosted Trees	maxDepth 4, maxIter 100

stage2_spark.py – preprocessor + tuning

```
stages = [Imputer(inputCols=NUMERIC, outputCols=imp,
                  strategy='median')]
stages += [StringIndexer(inputCol=c,
                        outputCol=f'{c}_idx',
                        handleInvalid='keep')
           for c in CATEGORICAL]
stages.append(OneHotEncoder(..., dropLast=True,
                           handleInvalid='keep'))
stages.append(VectorAssembler(outputCol='features',
                             inputCols=imp + ['bureau_data_missing'] + ohe))

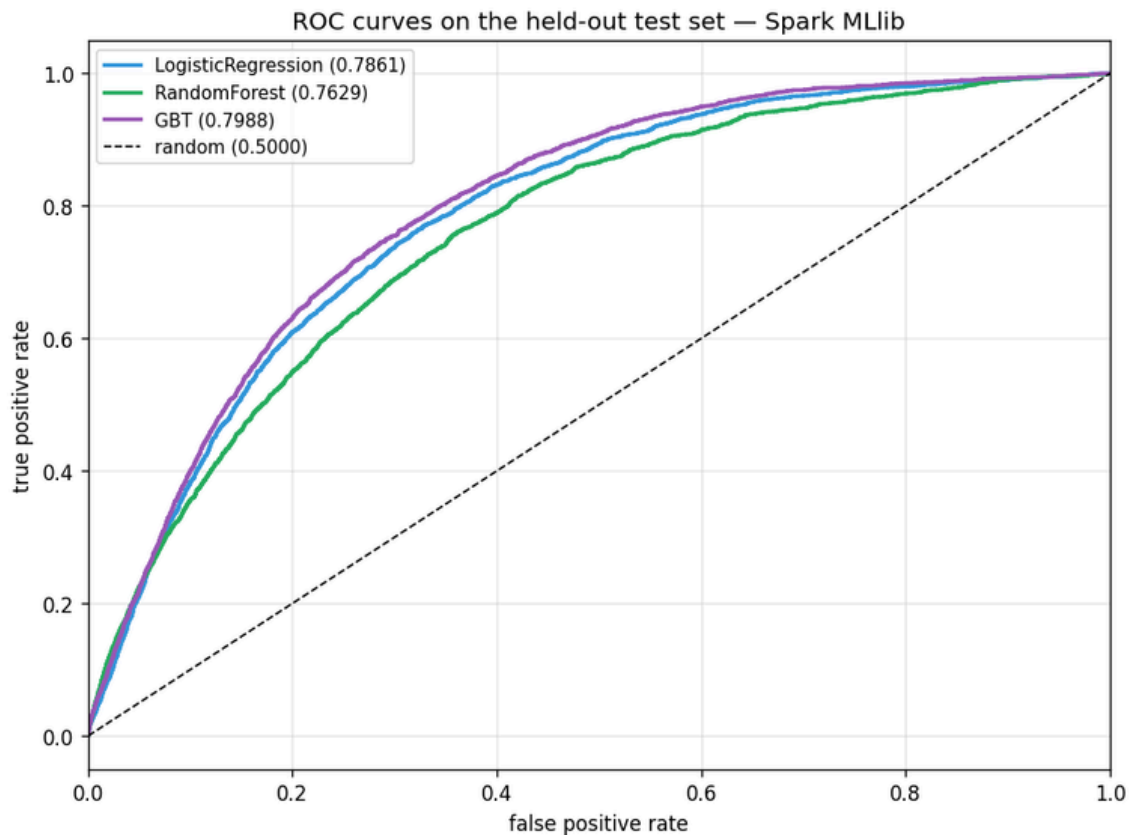
gbt = GBTClassifier(labelCol='default_ind',
                    weightCol='weight', seed=SEED)
cv = CrossValidator(estimator=gbt, numFolds=3,
                   evaluator=BinaryClassificationEvaluator(
                       metricName='areaUnderPR'))
```

MODEL CHOICE

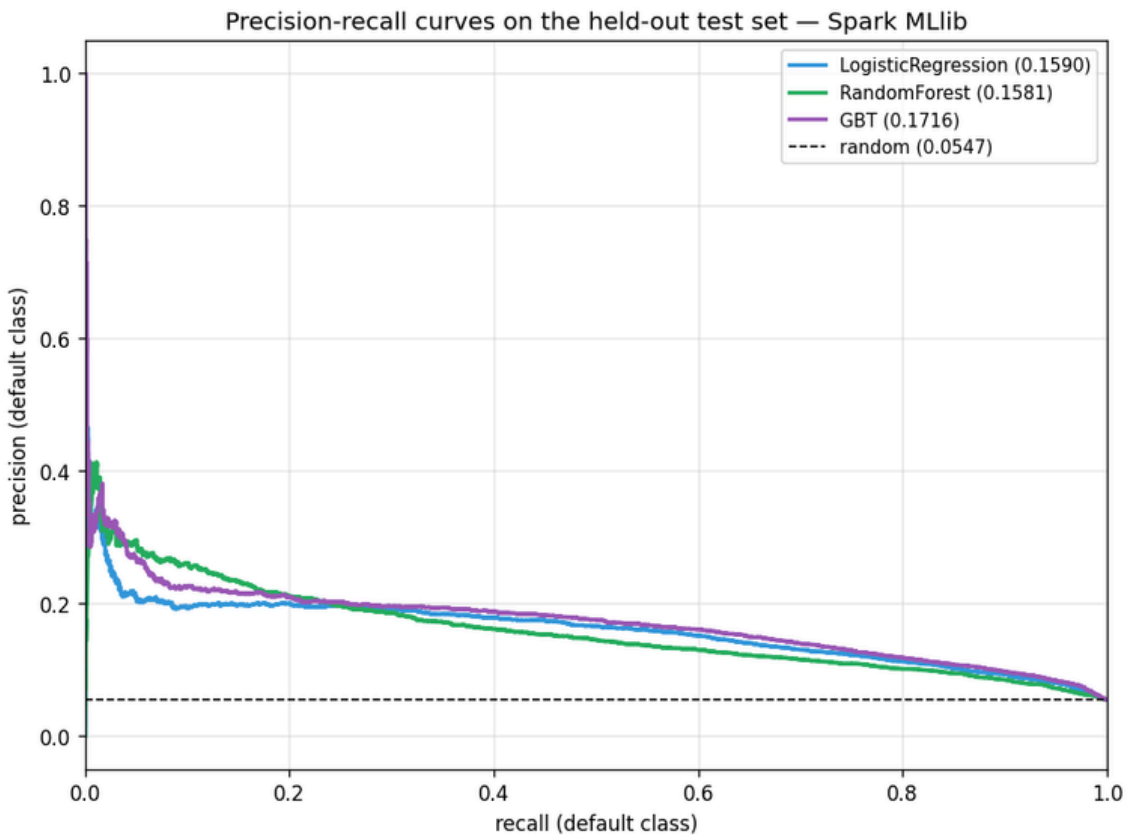
Why these three: a regularised linear baseline, a bagged non-linear model and a boosted one — increasing capacity on identical inputs, so any gain belongs to the model family.

Process One — Results on the Held-Out Test Set

Model	ROC-AUC	PR-AUC	Threshold	Recall (default)	Precision	F1 (default)	Accuracy
Logistic Regression	0.7861	0.1590	0.635	50.3%	16.5%	0.2490	83.4%
Random Forest	0.7629	0.1581	0.615	27.5%	19.0%	0.2245	89.6%
Gradient-Boosted Trees	0.7988	0.1716	0.680	45.0%	18.2%	0.2593	85.9%



ROC curves — test set

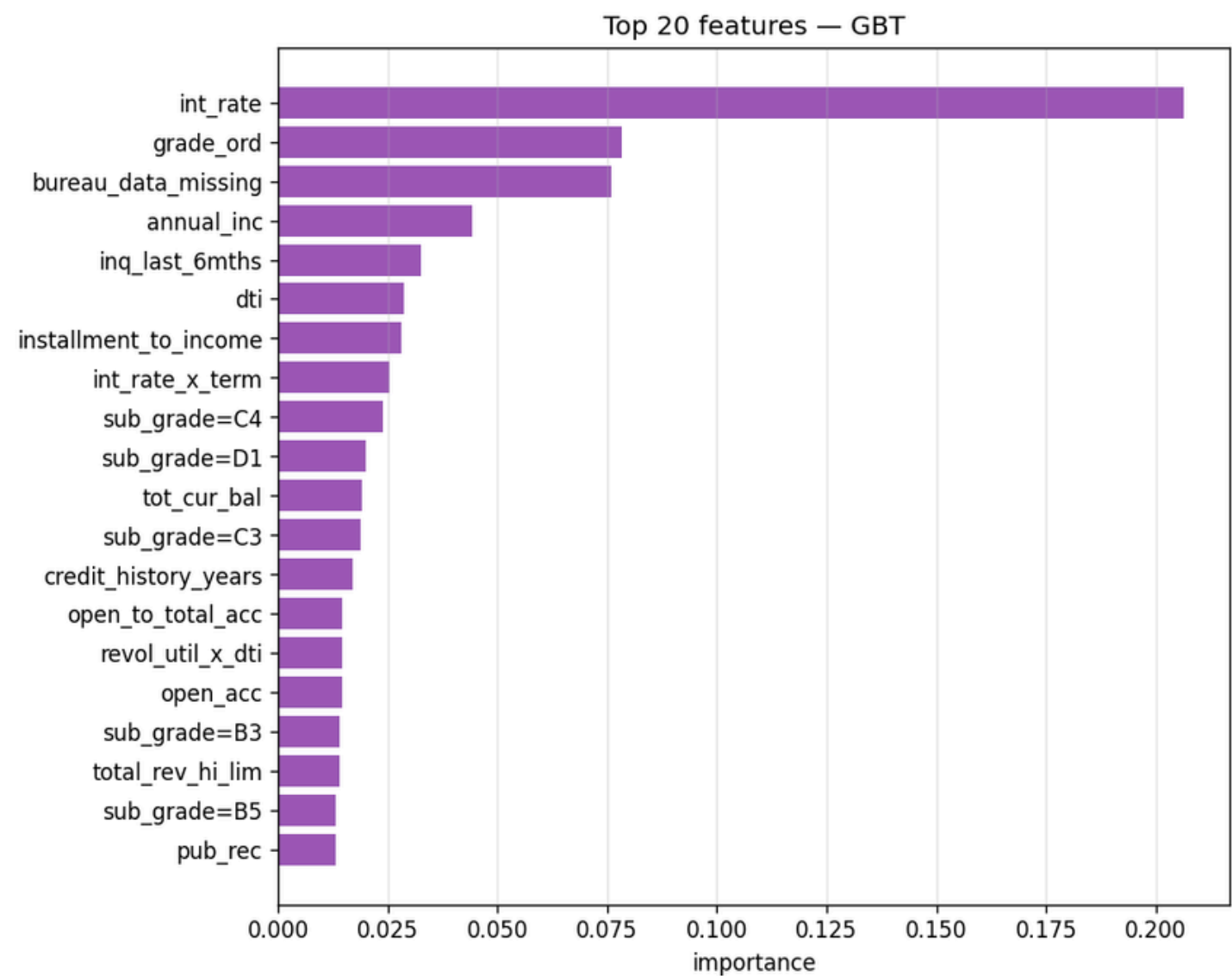


Precision-recall curves — test set

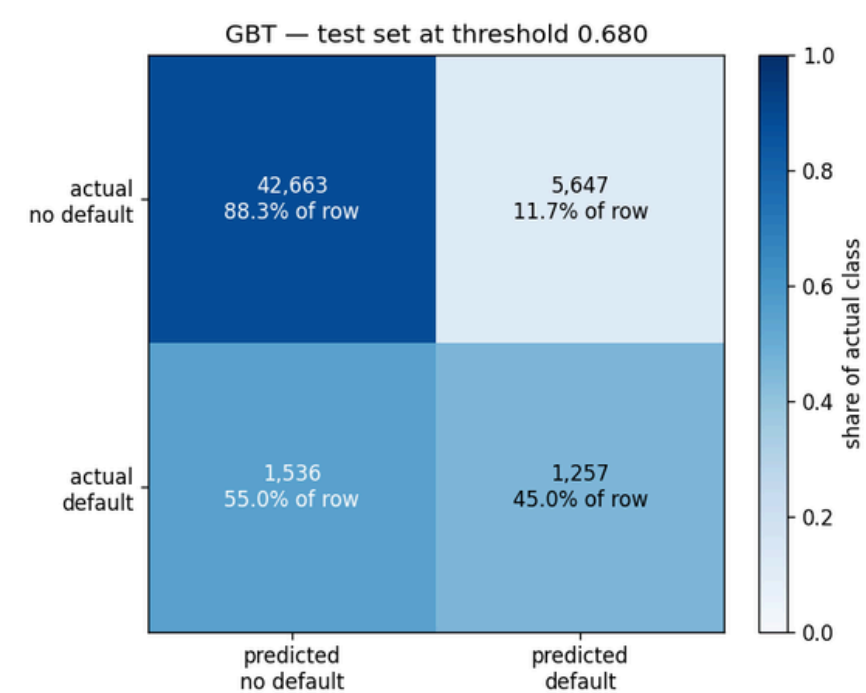
READ IT LIKE THIS

GBT wins on PR-AUC: 0.1716, or 3.14× the 5.47% random baseline. At its tuned threshold it finds 45.0% of defaults (1,257 of 2,793) while flagging 5,647 good loans. Random Forest has the best accuracy and the worst recall — the clearest evidence that accuracy is the wrong yardstick here.

What Drives the Best Spark Model



Gradient-Boosted Trees — top feature importances



GBT confusion matrix at threshold 0.680

int_rate, 20.7% One attribute carries a fifth of the total importance.
grade_ord, 7.8% The ordinal grade encoding earns its place.
bureau_data_missing, 7.6% The missingness flag is the third most important feature.
Then the ratios. annual_inc, inq_last_6mths, dti, installment_to_income and the int_rate × term interaction.

NEW INSIGHT

Whether a borrower has bureau data at all predicts default almost as strongly as their assigned grade. Missingness is not a nuisance to be imputed away — it is signal.

Process Two – TensorFlow / Keras

LogisticNN

137 parameters

A single sigmoid unit — a neural re-implementation of logistic regression, included as the capacity floor.

MLP

10,881 parameters

Dense 64 → dropout 0.3 → dense 32 → dropout 0.2 → sigmoid. Two hidden layers, ReLU.

DeepMLP

28,673 parameters

128 → BN → 0.4 → 64 → BN → 0.3 → 32 → 0.2 → sigmoid. Batch normalisation keeps the deeper stack trainable.

Same contract as Process One. The same clean file, the same 64 / 16 / 20 stratified fractions and the same threshold routine — 136 design-matrix columns after encoding.

Imbalance handled twice over. Class weight 17.31 on defaults, and the output bias initialised to $-2.8515 = \log(p/(1-p))$ so the network starts at the true base rate instead of spending its first epochs discovering that defaults are rare.

Early stopping on validation PR-AUC. Patience 8, best weights restored — not on loss, and not on accuracy.

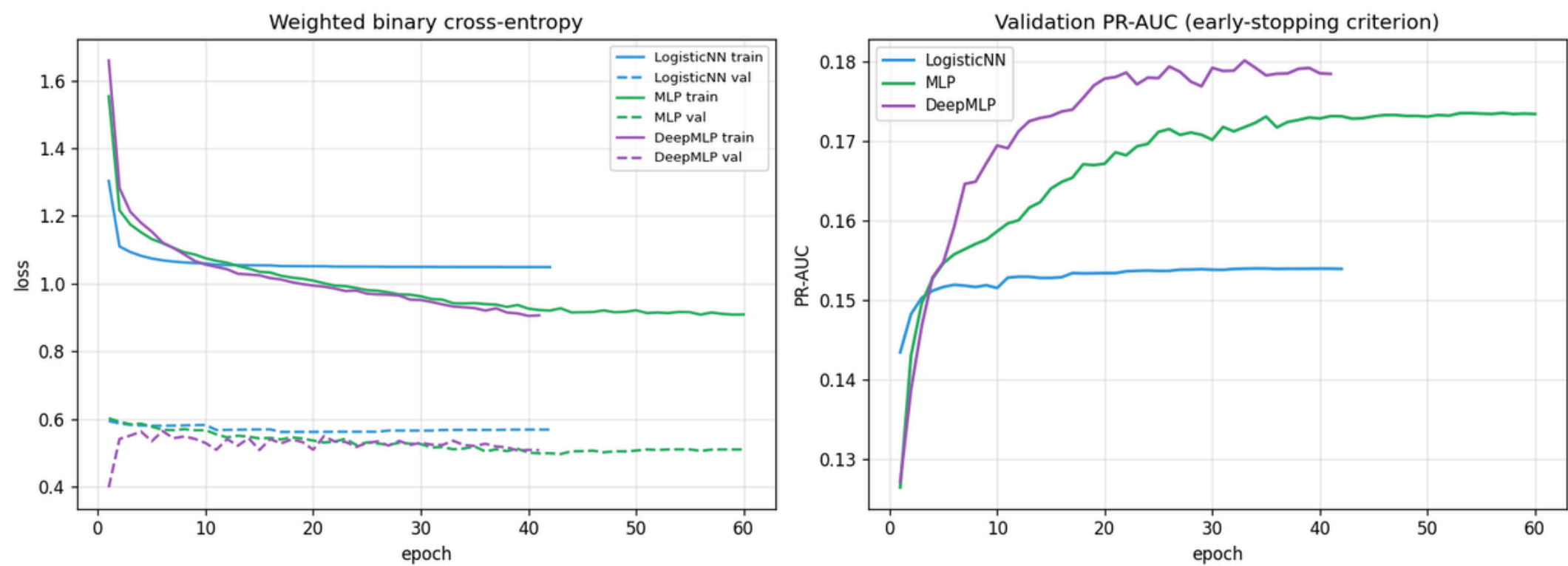
stage2_tensorflow.py – DeepMLP

```
def build_deep_mlp(n_features, bias):
    return keras.Sequential([
        layers.Input(shape=(n_features,)),
        layers.Dense(128, activation='relu'),
        layers.BatchNormalization(),
        layers.Dropout(0.4),
        layers.Dense(64, activation='relu'),
        layers.BatchNormalization(),
        layers.Dropout(0.3),
        layers.Dense(32, activation='relu'),
        layers.Dropout(0.2),
        # start at the true base rate
        layers.Dense(1, activation='sigmoid',
                    bias_initializer=Constant(bias)),
    ])
```

Three capacities on identical inputs isolate one question: how much non-linear signal is actually in these features?

Process Two – Results on the Held-Out Test Set

Model	Params	Epochs	ROC-AUC	PR-AUC	Threshold	Recall (default)	Precision	F1 (default)
LogisticNN	137	42	0.7881	0.1618	0.620	55.0%	16.2%	0.2502
MLP (64/32)	10,881	60	0.8025	0.1775	0.710	46.6%	17.8%	0.2579
DeepMLP (128/64/32)	28,673	41	0.8021	0.1830	0.750	43.6%	19.0%	0.2649



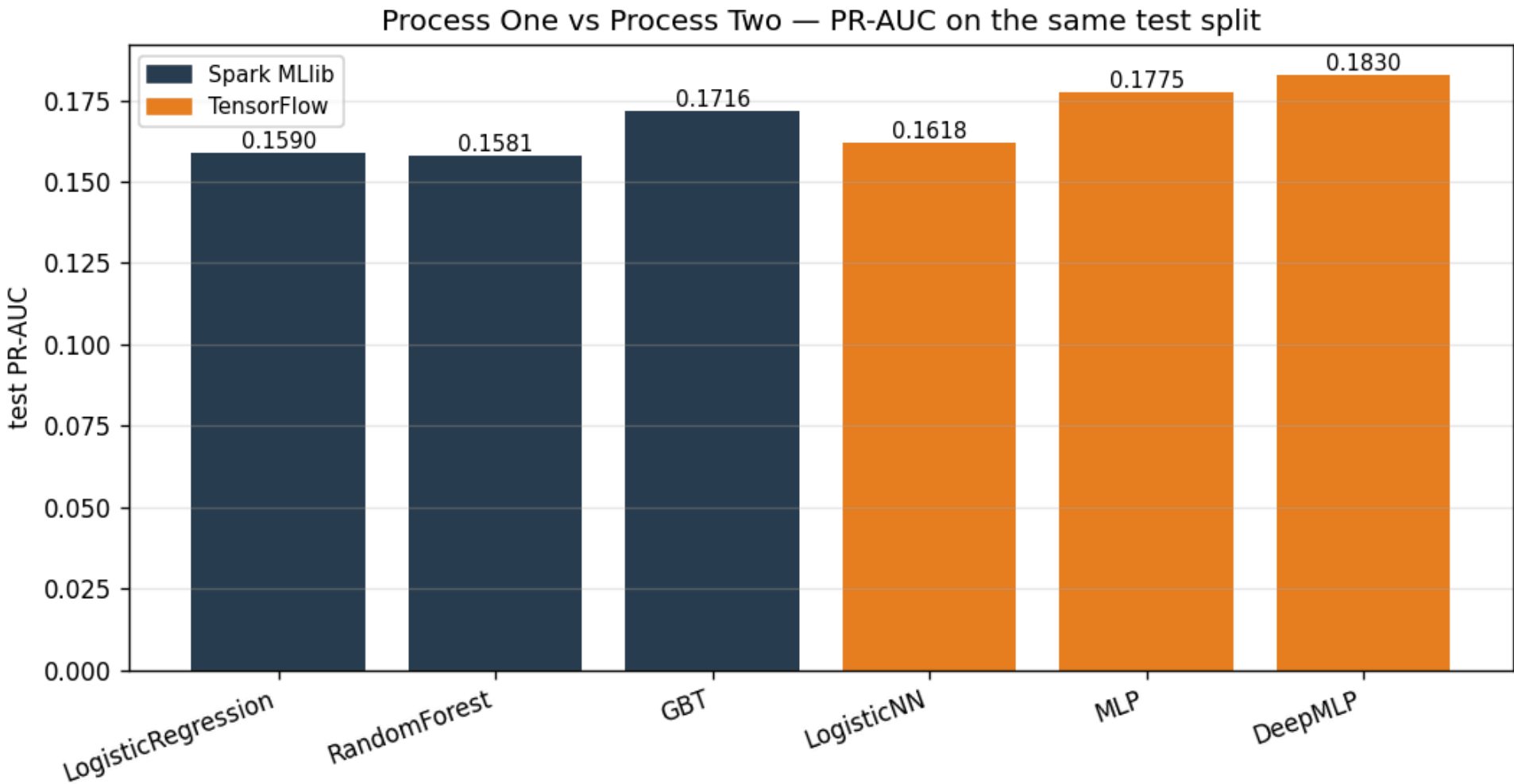
Training history – loss and validation PR-AUC, early stopping on PR-AUC

Depth is worth +0.0212 PR-AUC. DeepMLP 0.1830 against LogisticNN 0.1618 – that gap is a direct measurement of the non-linear signal in the features.

But it is not free. DeepMLP buys precision with recall: 19.0% precision at 43.6% recall, where the single-unit LogisticNN reaches 55.0% recall at 16.2% precision.

Early stopping did its job. Best epochs 34, 57 and 33 out of a 60-epoch budget; the deeper models start over-fitting well before the cap.

Head to Head – Spark MLlib vs TensorFlow



PR-AUC of all six models on the same test split — random baseline 0.0546

THE SPREAD IS NARROW

All six models sit between 0.158 and 0.183. The signal ceiling comes from the features, not from the library — and both stacks agree on what matters: `int_rate`, `grade` and the missingness flag.

Rank	Model	Process	PR-AUC
1	DeepMLP	TensorFlow	0.1830
2	MLP	TensorFlow	0.1775
3	GBT	Spark MLlib	0.1716
4	LogisticNN	TensorFlow	0.1618
5	LogisticRegression	Spark MLlib	0.1590
6	RandomForest	Spark MLlib	0.1581

WHAT THIS REALLY TELLS US

The honest conclusion is not “neural networks win”. Every model lands in the same narrow band, so the choice should be made on operational cost, interpretability and calibration — not on 0.01 of PR-AUC. DeepMLP does beat the best Spark model by +0.0113 PR-AUC, a real but modest 6.6% relative gain.

03

Critical Analysis

Strengths, weaknesses and what we would do differently

Strengths and Weaknesses of Our Approach

STRENGTHS

Leakage is controlled explicitly

13 post-funding columns are named and excluded, not merely overlooked.

One feature contract, two stacks

Both processes read the same clean file and the same split, so the comparison measures models, not preprocessing.

Imbalance handled end to end

Class weights, PR-AUC selection, a tuned threshold and an initialised output bias — four consistent decisions.

Test set touched once

Everything is chosen on validation; the reported numbers are not the best of many test peeks.

Reproducible

Fixed seeds, timestamped runs, runs.log and a pinned final/ directory behind every figure here.

WEAKNESSES

PR-AUC ≈ 0.18 is low in absolute terms

Good for ranking and pricing, not for an automatic decline.

Random split, not temporal

Loans span vintages; a deployed model predicts forward in time.

Stage 2 ran on a 30% stratified sample

256,861 rows; the full-data run is still outstanding.

The threshold optimises F1, not money

F1 weights a missed default and a rejected good borrower equally.

Partly circular features

int_rate and grade are Lending Club's own risk assessment.

Probabilities are uncalibrated

Tuned thresholds land at 0.62–0.75: the scores rank well but are not probabilities.

Lessons learned — choose the metric before the model, treat missingness as a feature, and fix split, weighting and threshold up front.

What We Learned

Missingness is a feature

bureau_data_missing is the 3rd most important input to the best Spark model, at 7.6%. A thin or absent credit file predicts default almost as strongly as the assigned grade does.

Accuracy is actively misleading

Random Forest posts the highest accuracy (89.6%) and the worst recall (27.5%). A constant “no default” rule beats every model on accuracy at 94.53%.

Ratios beat raw magnitudes

annual_inc alone reaches $r = -0.037$; installment_to_income reaches $+0.062$. Engineering the affordability ratio extracted signal no new data could.

Capacity buys little here

137 parameters → 28,673 parameters gains $+0.021$ PR-AUC. The features, not the architecture, are the binding constraint.

The two stacks converge

Six models across two independent libraries land within 0.025 PR-AUC of each other — strong evidence we are near the ceiling of this feature set.

Risk is priced, then mispriced

Grade separates 1.83% from 16.70% default, so the lender's own model is informative — but it still leaves 45% of defaults recoverable by a better one.

Bottom line — on origination-only information the best model recovers roughly 45% of defaults at a $3.35\times$ lift over chance. That is a genuinely useful risk-ranking tool and a genuinely unusable automatic underwriter, and the difference between those two claims is the whole project.

Conclusions and Future Work

We delivered a complete, leakage-free workflow: Spark exploration over all 855,969 loans, six models across two independent stacks, and a single evaluation protocol shared by both. The best model, a DeepMLP, reaches PR-AUC 0.183 — 3.35× the random baseline — and recovers 43.6% of defaults at its tuned threshold.

WHAT WE WOULD DO NEXT

1

Temporal validation

Split by issue_d — train on earlier vintages, test on later ones. The realistic deployment test, and it will lower the numbers.

2

Cost-sensitive thresholds

Replace F1 with expected loss: loss-given-default against forgone interest. The threshold should come from money, not from a symmetric metric.

3

Probability calibration

Isotonic or Platt scaling so the scores can be read as default probabilities and fed into pricing.

4

Full-data Stage 2

Re-run both processes on all 855,969 rows to confirm the 30%-sample ranking holds.

5

Interpretability

SHAP values on the GBT to give per-application reason codes — a regulatory requirement for real credit decisions.

6

Ensembling and fairness

Blend the GBT and DeepMLP scores, and audit default rates by addr_state and emp_length for disparate impact.

Thank you

Questions?

Every figure and table in this deck is written out by `src/stage1_eda.py`, `src/stage2_spark.py` and `src/stage2_tensorflow.py` into `results/*/final/` — the pinned runs behind these slides.

REPRODUCE IT

```
python src/stage1_eda.py
python src/stage2_spark.py
python src/stage2_tensorflow.py
PySpark 4 · TensorFlow 2.19 · Python 3.12 · JDK
17
```